

10. Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity.

OUTPUT :

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>

int main()
{
    int fd;
    struct stat st;
    char *data;

    fd = open("data.txt", O_RDWR);
    if (fd == -1)
    {
        perror("open");
        return 1;
    }

    if (fstat(fd, &st) == -1)
    {
        perror("fstat");
        close(fd);
        return 1;
    }

    data = mmap(NULL,
                st.st_size,
                PROT_READ | PROT_WRITE,
                MAP_SHARED,
                fd,
                0);

    if (data == MAP_FAILED)
    {
        perror("mmap");
        close(fd);
        return 1;
    }

    printf("Original file contents:\n");
    write(STDOUT_FILENO, data, st.st_size);

    /* Modify the mapped file */
    if (st.st_size >= 5)
    {
        memcpy(data, "HELLO", 5);
    }

    msync(data, st.st_size, MS_SYNC);

    printf("\nFile modified using mmap().\n");

    munmap(data, st.st_size);
    close(fd);

    return 0;
}
```

```
hemaswi@Hemaswi:~$ nano data.txt
hemaswi@Hemaswi:~$ nano practical10.c
hemaswi@Hemaswi:~$ gcc practical10.c -o practical10
hemaswi@Hemaswi:~$ ./practical10
Original file contents:
This is my data file.

File modified using mmap().
hemaswi@Hemaswi:~$ |
```

11 . Develop a multithreaded counter application using POSIX threads (`pthread_create`, `pthread_join`). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

Modify the previous program using mutex locks (`pthread_mutex_lock`, `pthread_mutex_unlock`) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

OUTPUT :

```

#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 1000000

int counter = 0;

void *increment(void *arg)
{
    for (int i = 0; i < ITERATIONS; i++)
    {
        counter++;
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_create(&threads[i], NULL, increment, NULL);
    }

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("Expected value: %d\n",
        NUM_THREADS * ITERATIONS);

    printf("Actual value: %d\n", counter);

    return 0;
}

```

```

hemaswi@Hemaswi:~$ nano practical11A.c
hemaswi@Hemaswi:~$ gcc practical11A.c -o practical11A
hemaswi@Hemaswi:~$ ./practical11A
Expected value: 4000000
Actual value: 1333938
hemaswi@Hemaswi:~$

```

11(B):

```

#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 1000000

int counter = 0;

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *increment(void *arg)
{
    for (int i = 0; i < ITERATIONS; i++)
    {
        pthread_mutex_lock(&lock);

        counter++;

        pthread_mutex_unlock(&lock);
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_create(&threads[i], NULL, increment, NULL);
    }

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("Expected value: %d\n",
           NUM_THREADS * ITERATIONS);

    printf("Actual value: %d\n", counter);

    pthread_mutex_destroy(&lock);

    return 0;
}

```

```

hemaswi@Hemaswi:~$ nano practical11B.c
hemaswi@Hemaswi:~$ gcc practical11B.c -o practical11B
hemaswi@Hemaswi:~$ ./practical11B
Expected value: 4000000
Actual value: 4000000

```